INFORME EJECUTIVO Y TÉCNICO DE PENTESTING



Evaluación de Seguridad Ofensiva, Hackeo Ético
y Verificación OWASP ASVS v4.0.3

Índice / Tabla de Contenidos

1. Resumen Ejecutivo y Ficha Técnica Pág. 3

2. Metodología de Evaluación y Estándares Aplicados Pág. 4

2.1 Marcos Normativos Utilizados Pág. 4

2.2 Escala de Severidad y Prioridad Pág. 4

3. Métricas y Distribución Estadísticas de Hallazgos Pág. 5

Gráficos de Distribución (Barras — Estado del Ticket) Pág. 6

Gráficos de Distribución (Torta — Severidad) Pág. 7

4. Alcance Técnico y Componentes Evaluados Pág. 8

5. Detalle de Vulnerabilidades e Incidencias Registradas Pág. 9

Prueba de Concepto (PoC) paso a paso Pág. 10

Evidencias Fotográficas (Capturas de Pantalla) Pág. 11

6. Conclusiones y Plan de Remediación Recomendado Pág. 12

1. Resumen Ejecutivo y Ficha Técnica

Durante la evaluación del laboratorio «Asistencias» (Módulo II, Objetivo 2) detecté que el formulario de verificación de código responde en tiempos marcadamente distintos según si el código ingresado corresponde a un asistente registrado (~1 s) o no (~0,2 s), sin necesidad de conocer la clave.

Esa diferencia de tiempo es un canal lateral (timing side-channel) que permite enumerar qué códigos tienen una cuenta asociada y así deducir el total de asistentes (159) sin autenticarse. Severidad: MEDIA (CVSS 5.3). Se recomienda implementar comparación en tiempo constante ejecutar siempre el hash de la clave, incluso para códigos inexistentes y sumar rate limiting.

Parámetro / Campo	Detalle de la Evaluación
Tipo de Evaluación	Penetration Testing (Caja Gris) — Enumeración de usuarios por canal de tiempo
Estándar Principal	OWASP ASVS 4.0.3 (Level 2/3) & OWASP Top 10
Período de Pruebas	Octubre 2026 — Laboratorio de práctica (Módulo II · Objetivo 2)
Entorno de Evaluación	Ambiente de Laboratorio — Aplicación «Asistencias»
Clasificación del Documento	**ESTRICTAMENTE CONFIDENCIAL**

2. Metodología de Evaluación y Estándares Aplicados

Las pruebas se ejecutaron combinando análisis estático (SAST), análisis dinámico (DAST) y pruebas de penetración manuales profundas, para reducir falsos positivos y garantizar la exhaustividad del análisis.

2.1 Marcos Normativos Utilizados

OWASP ASVS v4.0.3

Verificación estructurada de controles de seguridad en arquitectura, autenticación, control de acceso y criptografía.

OWASP Top 10

Clasificación de las vulnerabilidades web más críticas: inyecciones, fallos de control de acceso, criptografía deficiente, entre otras.

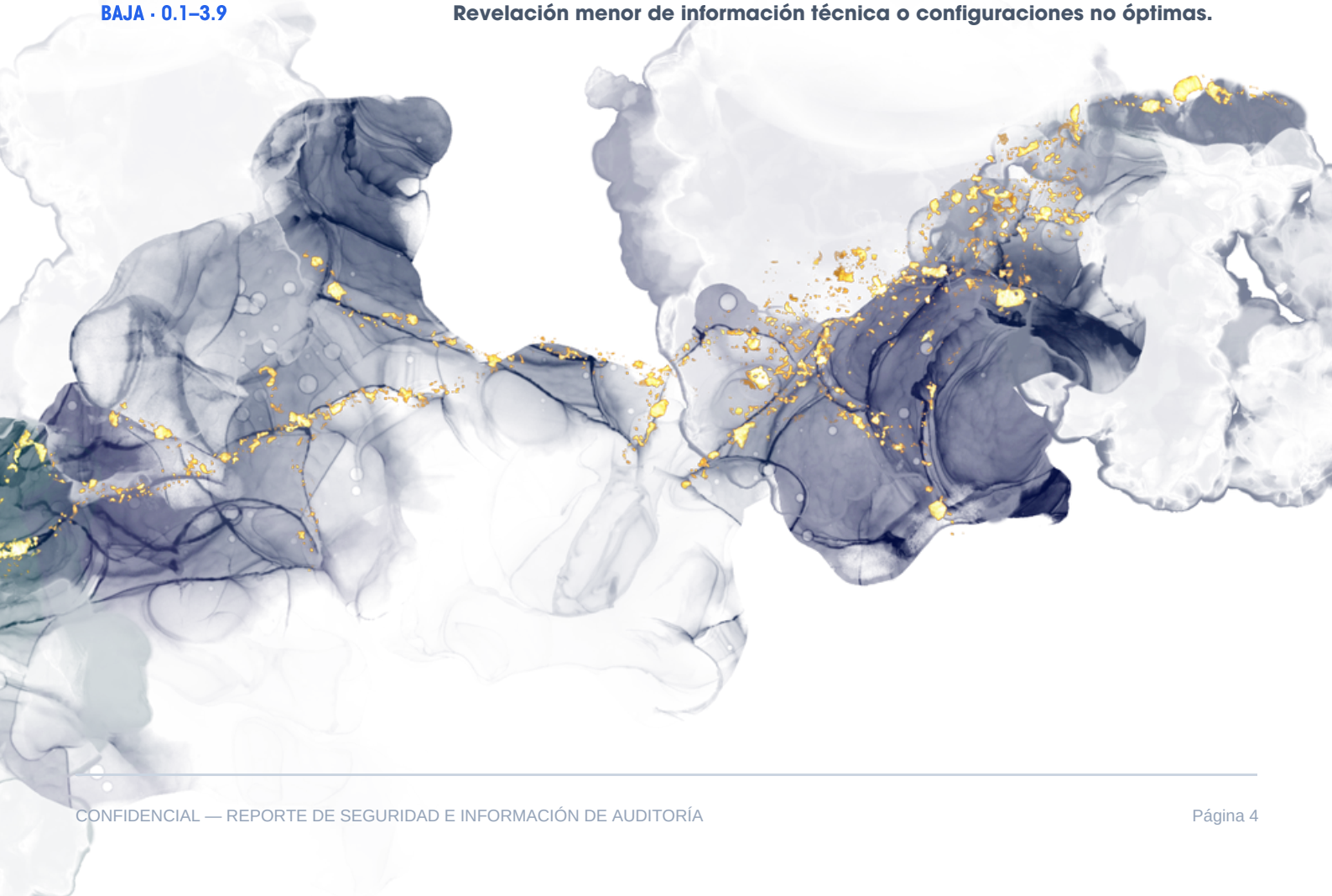
PTES — Penetration Testing Execution Standard

Guía de fases del pentest: reconocimiento, descubrimiento, explotación y post-explotación.

2.2 Escala de Severidad y Prioridad (CVSS v3.1)

Los hallazgos se evalúan bajo la matriz CVSS v3.1, clasificándose en cuatro niveles:

CRÍTICA · 9.0–10.0	**Ejecución remota de código o acceso total al sistema sin autenticación.**
ALTA · 7.0–8.9	**Compromiso grave de confidencialidad o integridad con impacto sustancial.**
MEDIA · 4.0–6.9	**Fallos de lógica o bypass de controles que requieren condiciones previas.**
BAJA · 0.1–3.9	**Revelación menor de información técnica o configuraciones no óptimas.**



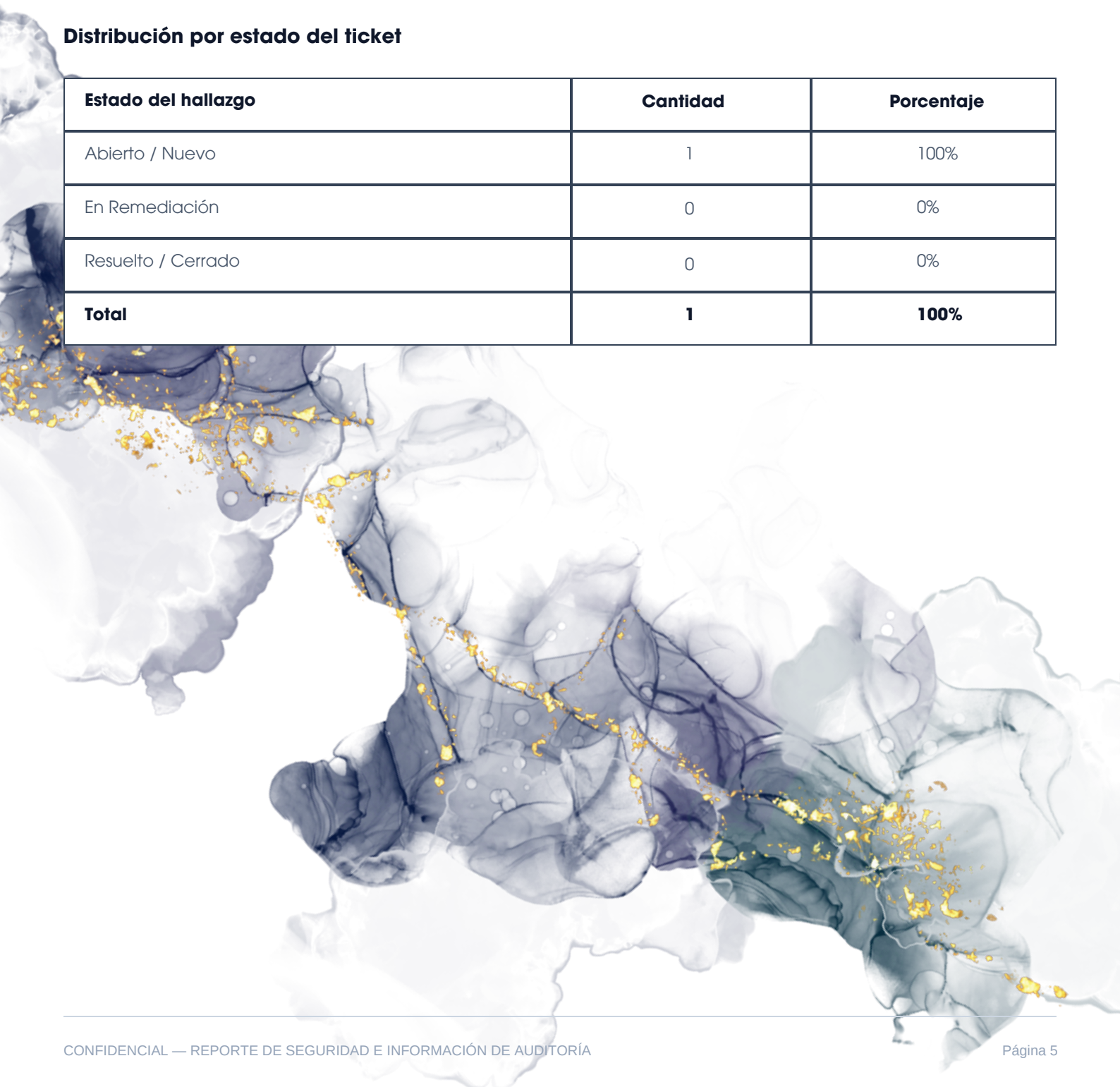
3. Métricas y Distribución Estadística de Hallazgos

A continuación se consolidan las métricas del hallazgo registrado durante esta auditoría puntual (laboratorio Asistencias — enumeración de usuarios por canal lateral de tiempo).

1	0	1	0
Hallazgos totales	Severidad Alta	Severidad Media	Severidad Baja

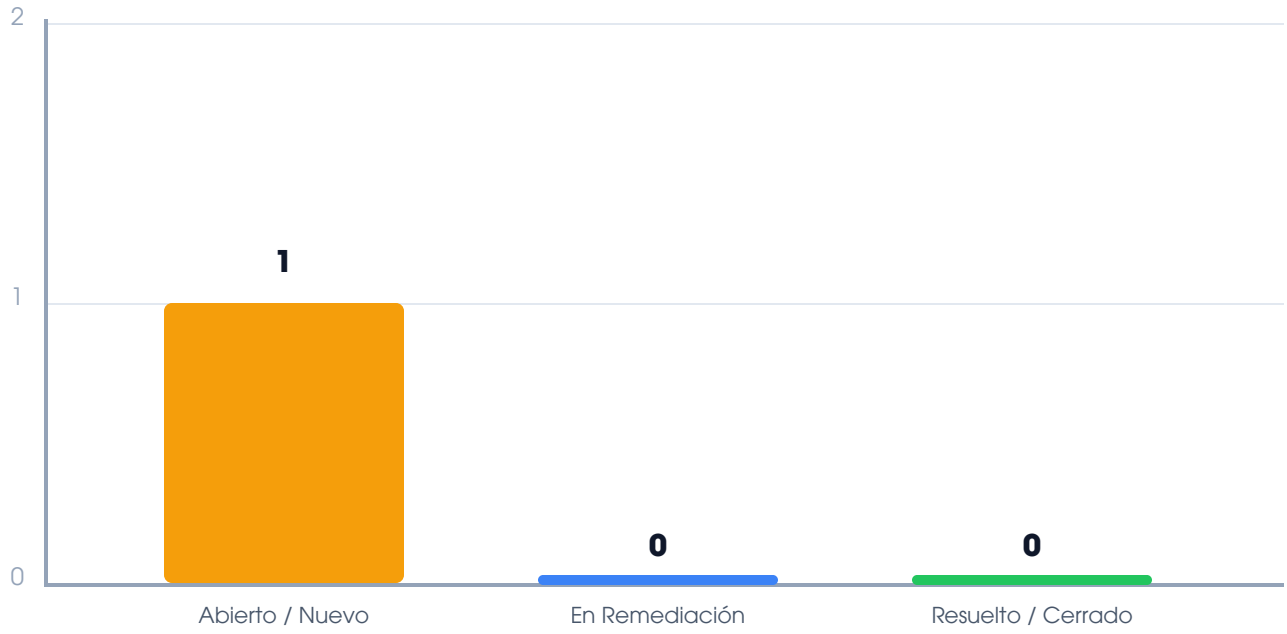
Distribución por estado del ticket

Estado del hallazgo	Cantidad	Porcentaje
Abierto / Nuevo	1	100%
En Remediación	0	0%
Resuelto / Cerrado	0	0%
Total	**1**	**100%**



Distribución por Estado del Ticket

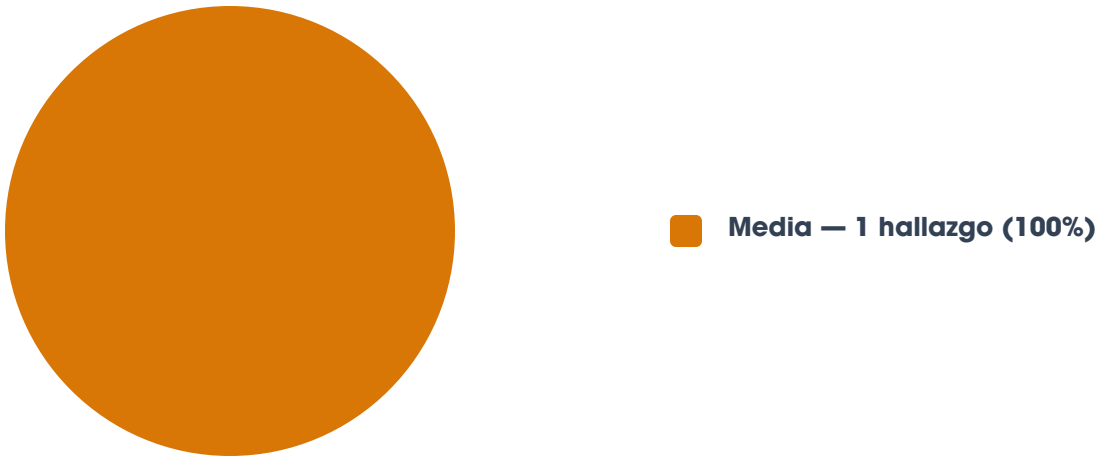
El gráfico de barras muestra cómo se reparten los hallazgos según su estado de gestión al cierre de la auditoría.



Cada barra representa la cantidad de hallazgos en ese estado. El único hallazgo de esta auditoría se encuentra actualmente Abierto, pendiente de remediación.

Distribución por Severidad

La torta muestra la proporción de hallazgos según su nivel de severidad CVSS.



El único hallazgo de esta auditoría es de severidad Media (CVSS 5.3) y concentra el 100% del riesgo identificado.

4. Alcance Técnico y Componentes Evaluados

Esta auditoría se centró en la autenticación del formulario de verificación de código de «Asistencias», probada de forma activa (envío de peticiones y medición de tiempos de respuesta). A continuación, qué se evaluó y qué quedó fuera de alcance:

Autenticación y Enumeración de Usuarios — Evaluado

El endpoint POST /index.php responde en tiempos distintos según si el código existe (~1 s) o no (~0,2 s), lo que permite enumerar cuentas sin conocer la clave — origen del hallazgo VULN-001.

Control de Acceso e Impersonación — Fuera de alcance

No se evaluó la lógica de autorización entre usuarios; el foco fue la enumeración de cuentas por canal de tiempo.

Seguridad en Comunicaciones y Criptografía — Fuera de alcance

No se evaluaron en esta auditoría puntual los atributos de cookies, el TLS ni las cabeceras de hardening HTTP.

Manejo de Errores y Registro de Eventos — Evaluado (sin hallazgo)

El formulario devuelve siempre el mismo mensaje genérico («No se puede acceder a esta información»), sin stack trace ni datos internos expuestos.

5. Detalle de Vulnerabilidades e Incidencias Registradas

Se detalla el hallazgo principal identificado, con su descripción técnica, clasificación normativa y desglose de severidad CVSS.

VULN-001 · Enumeración de usuarios por canal lateral de tiempo **MEDIA · CVSS 5.3**

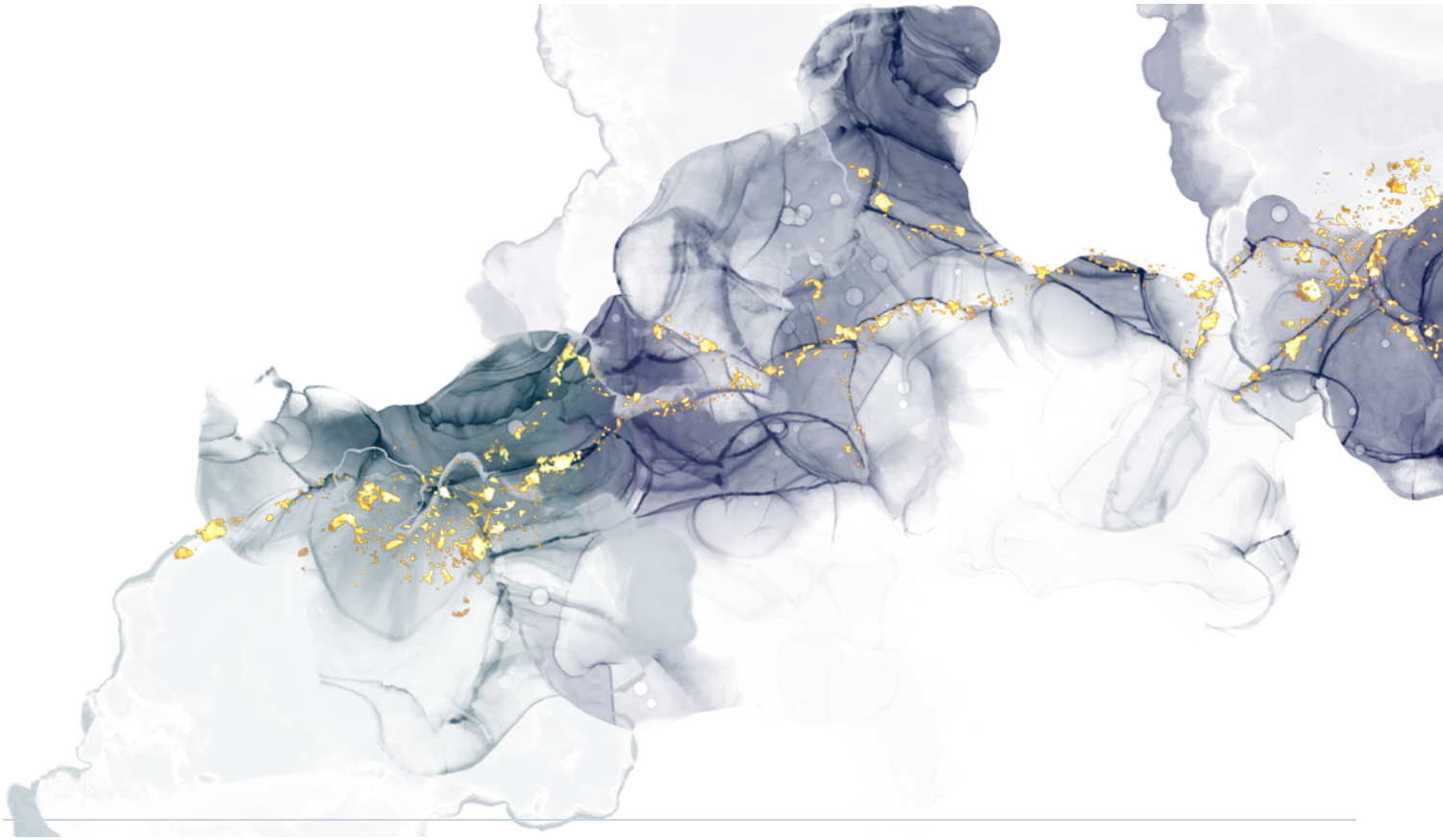
CWE	**CWE-208** Observable Timing Discrepancy (relacionado con CWE-204, enumeración de usuarios)
OWASP Top 10	**A07:2021** — Identification and Authentication Failures
Vector CVSS 3.1	**AV:N / AC:L / PR:N / UI:N / S:U / C:L / I:N / A:N**
Estado	**Abierto**

Descripción técnica

El formulario de verificación («codigo» + «clave») compara la clave contra la cuenta asociada al código. Cuando el código existe, el servidor procesa esa comparación y tarda ~1 s; cuando el código no existe, corta de inmediato y responde en ~0,2 s. Esa diferencia de tiempo, estable y repetible, revela qué códigos tienen cuenta asociada sin necesidad de conocer la clave.

Impacto

Un atacante puede recorrer las 1000 combinaciones de código (000–999) midiendo el tiempo de respuesta y enumerar todas las cuentas válidas, deduciendo la cantidad total de asistentes (159) y qué códigos están en uso, sin autenticarse ni disparar bloqueos. Es una fuga de información que facilita ataques dirigidos posteriores.



Prueba de Concepto (PoC) — Paso a Paso

Reproducción completa del hallazgo en el laboratorio Asistencias, de forma activa (envío de peticiones y medición de tiempos de respuesta). Cada paso es reproducible y verificable.

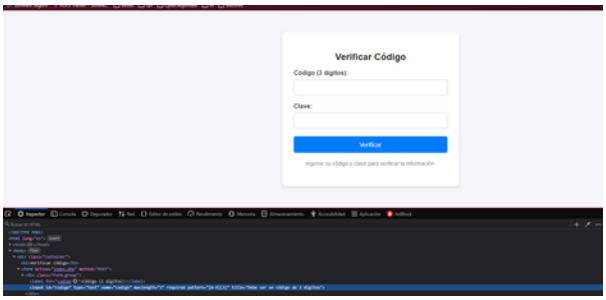
1. Abrir el formulario de verificación y notar en el HTML el campo codigo con pattern="(0-9){3}": 3 dígitos = 1000 combinaciones (000–999).
2. Inspeccionar la petición, envia código sin clave, nos da pie a una automatización.
3. Configurar en el Intruder un ataque para verificar diferencias, se verifica que algunos tardan ~1 s y otros ~0,2 s. El retardo depende del código, y es repetible.
4. Automatizar con un script que recorre codigo 000–999 (clave fija) y mide el tiempo de respuesta de cada POST /index.php.
5. Fijar el umbral DENTRO de ese hueco (~0,8 s), no en un valor fijo arbitrario, y contar los códigos que lo superan. Validar repitiendo la corrida: el resultado se mantiene.
6. **Resultado: 159 códigos lentos = 159 asistentes. MD5, el código que resuelve el desafío.**

```
# Timing attack - clave fija, variar codigo (000-999)
asistentes = 0
for n in range(1, 1000):
    codigo = f"{n:03d}"
    t0 = time.perf_counter()
    session.post(URL, data={"codigo": codigo, "clave": "gochip"})
    seg = time.perf_counter() - t0
    if seg > 0.8:            # umbral en el hueco real de los datos
        asistentes += 1     # codigo con cuenta asociada

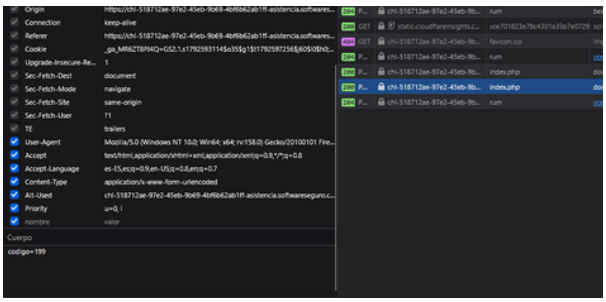
# asistentes = 159  ->  md5("159")
```

Evidencias

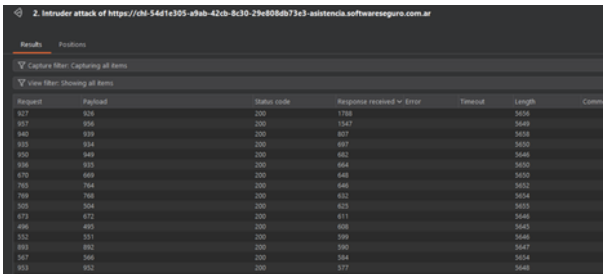
Capturas obtenidas durante la ejecución del ataque de tiempo. Cada imagen documenta un paso del PoC.



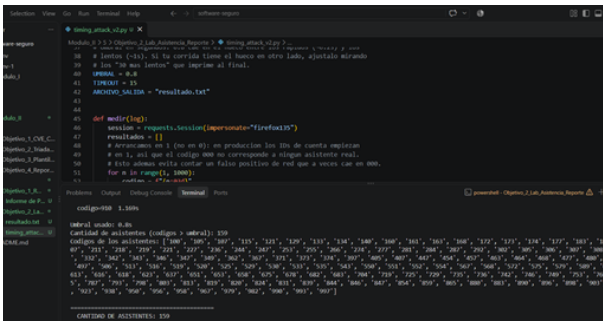
1. HTML del formulario - el campo codigo tiene pattern="[0-9]{3}": 3 dígitos, 1000 combinaciones posibles (000–999).



2. Inspeccionar la petición POST, envia código sin clave, nos da pie a una automatización.



3 · Configurar en el Intruder un ataque para verificar diferencias, se verifica en el response received.



4 · Automatizar con un script que recorre codigo 000–999 (clave fija) y mide el tiempo de respuesta de cada POST /index.php.

6. Conclusiones y Plan de Remediación Recomendado

Se recomienda priorizar la corrección del hallazgo Medio antes del pase a producción. A continuación, la corrección técnica y el plan de verificación.

Corrección recomendada — Comparación en tiempo constante + rate limiting

```
// Siempre ejecutar el hash de la clave, exista o no el codigo
$usuario  = buscarPorCodigo($codigo);
$hashReal = $usuario ? $usuario->hash : HASH_DUMMY; // hash señuelo
$ok = password_verify($clave, $hashReal);           // tiempo constante

if ($usuario && $ok) { /* acceso concedido */ }

// + rate limiting por IP para frenar el barrido de 1000 codigos
```

Plan de acción y timeline

INMEDIATO (0–24 h)	Ejecutar siempre la verificación de la clave (con un hash señuelo) aunque el código no exista, para igualar los tiempos.
CORTO PLAZO (1–3 d)	Sumar rate limiting / bloqueo por IP para impedir el barrido de las 1000 combinaciones de código.
VERIFICACIÓN (3–5 d)	Re-test de tiempos: medir códigos válidos vs. inexistentes y confirmar que ya no hay diferencia.

Criterios de verificación (re-test)

- El tiempo de respuesta es igual para códigos válidos e inexistentes (sin diferencia medible).
- El barrido de 000–999 se bloquea o limita por rate limiting antes de completarse.
- Ya no es posible deducir la cantidad de asistentes a partir del tiempo de respuesta.

Elaborado por
María de los Ángeles Acuña
QA Engineer

Revisado y aprobado por
Lead Security Auditor / CISO
Firma: _____



